

Arduino Lab #3: Sensors and Actuators

In this lab a light sensor and a servo motor actuator is combined with an Arduino. This allows the Arduino to move the servo motor to a desired angle and to measure incoming light.

Sensors and actuators

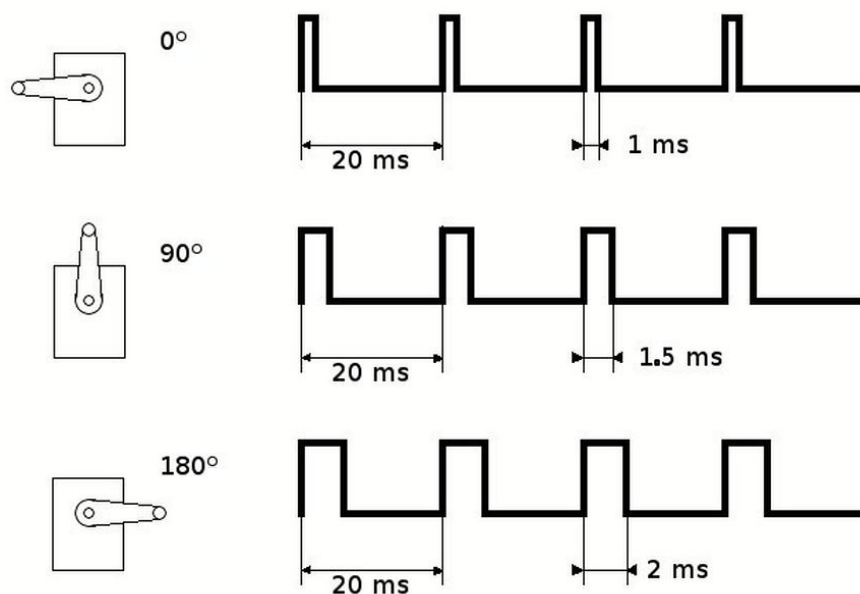
A **sensor** is a device used in a mechatronic system in order to measure some quantity in the system (e.g. position). An **actuator** is a device used to change the system (e.g. a DC motor). The sensors are analogous to the five senses of a person and the actuators are like the muscles and limbs. Actuators and sensors normally have electronics (combined or sold separately) to interface with a microcontroller. The microcontroller can be programmed to read the sensors and based on this information decide how to change/write the actuators. This gives the mechatronic system the capability to react to its environment and accurately control its response.

In this lab a micro servo motor actuator is used similar to the following items:

<https://www.robotshop.com/ca/en/dfrobot-micro-servo-motor.html>

<https://www.robotshop.com/ca/en/9g-micro-servo-motor-4-8v.html>

A servo motor has a built in controller plus amplifier electronics that adjust the voltage of the DC motor in order to achieve the desired position (sent from the Arduino). The servo receives a digital output signal from the Arduino in the form of a pulse (i.e. a 5 V / high signal) every 20 ms. The pulse width indicating the desired position varies from 1 ms (0 deg) to 2 ms (180 deg) (see figure below). The exact range of pulse width depends on the model.



The Arduino is capable of generating such pulses using a servo library function. The user simply has to call the servo function with the desired angle. It should be noted that the details of a servo as indicated above are not essential to the lab. The user can simply consider the servo as a device that attempts to go to a desired position set by an Arduino servo function.

Servo actuators are useful for moving / positioning parts in a mechatronic system such as robot links and robot wheels (if suitably modified for continuous rotation). Servos come in a wide variety of sizes, torque capacities, and speeds. They are also available with different gear types (nylon, metal, Karbonite, titanium, etc.). Other types of actuators such as brushless motors used in flying drones also use a servo type pulse interface.

The light sensor used in this lab is similar to the iTeaD Studio Electronic Brick Light Sensor described in the following links:

ftp://imall.iteadstudio.com/Electronic_Brick/IM120710017/DS_IM120710017.pdf

https://www.itead.cc/wiki/Light_Sensor_Brick

This type of sensor produces an analog (i.e. continuous) voltage that changes depending on the amount of incident light. The voltage varies continuously from 0 V to 5 V. It can be read using an analog input pin on the Arduino with the analog read library function.

Light sensors are typically used for obstacle detection (light is blocked when close to another object) and distance measurement. In order to measure distance a light source is needed. This light source can be mounted on a target object to measure its distance as shown in the lab example. The light received will vary depending on the distance. The relationship will not be directly proportional, but it can be calibrated using another sensor such as a measuring tape.

Wiring / connecting the components to the Arduino

Electronics is beyond the scope of this course, but a certain amount of wiring is needed in order to connect the components to the Arduino. Just carefully follow the wiring instructions below. However, students should try to remember the good wiring practices and tips discussed in this lab since they will be useful in the future and possibly save you money.

Wiring is a necessary evil with mechatronics, but it should be taken seriously. Incorrect wiring can damage your equipment (especially the Arduino), produce incorrect/inconsistent programming results, and waste a lot of time later on. ***Always check your wiring first*** before continuing with the programming task (or looking for programming errors) since you can waste a lot of time looking for programming errors when the actual problem is wiring (incorrect wiring, loose wires, cables not connected, power not connected, etc.). This advice is especially applicable to students without much electronics experience since rookie errors are very common at first.

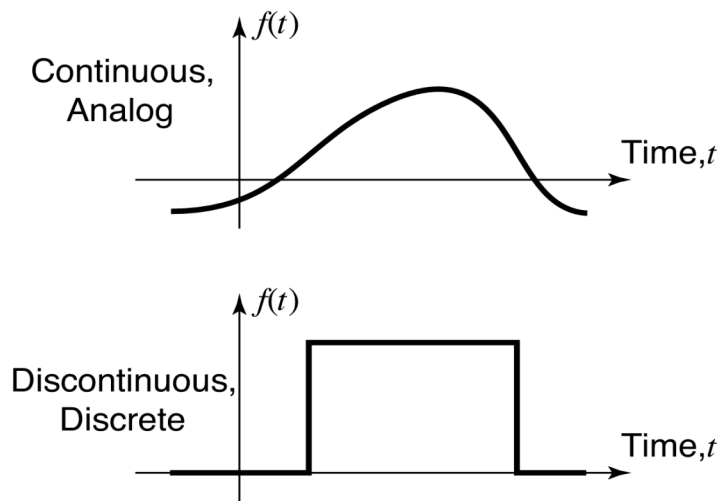
A detailed step by step guide showing each set of wiring connections (along with general wiring advice) is given by the following pdf file:

Lab Tip: Always disconnect the Arduino from the USB port of the PC (i.e. its power supply) when making new wiring connections with components. This helps prevent accidental damage of components and the Arduino.

Lab Tip: It pays to be careful with wiring in general for mechatronics projects. Your equipment could be damaged and your program won't work if the wiring is wrong. It can be very difficult to find wiring errors because they can look like programming errors or equipment malfunctions. I always triple check my wiring before proceeding with programming. This includes obvious things such as connecting the power supply to the outlet and the Arduino to the USB port.

Background and Definitions

In this lab an analog input is used (pin A0) in order to interface with the light sensor. The light sensor basically produces a voltage between 0V and 5V that changes in a continuous manner with the amount of light it receives. For example, it might produce a voltage of 1.5V a distance of 5 cm from a light bulb (high light), a voltage of 2.5V a distance of 20 cm (medium light), and a voltage of 4.1V a distance of 100 cm (low light). Notice that this is not a linear relationship, but voltage is a continuous function of distance. This type of continuous sensor is known as an **analog** sensor. Sensors that only provide discrete / discontinuous values such as on/off mechanical switches are known as **digital** sensors. The following figure illustrates the difference between the two types of sensors.



In Lab #2 digital input pins were introduced as a way to measure on/off digital voltages. Digital inputs cannot accurately measure analog voltages because there are only two states which are measured: LOW (0V) and HIGH (5V). Therefore, a different type of input is required. The Arduino board has 8 analog input pins (A0 to A7), often called Analog to Digital Conversion (ADC) pins, that are capable of measuring continuous / analog voltages. The default voltage range for these inputs is 0V to 5V, but it can be adjusted for smaller ranges using a library function. The voltage from the sensor is connected to one of the analog input pins. The ground of the sensor is connected to the GND pin of the Arduino (forming a common ground / reference). The Arduino analog input function returns integer values from 0 to 1023 (10-bit resolution) directly proportional to the input voltage (0 to 5V).

Note that it takes approximately 100 us to read an analog input. Thus, relatively fast analog signals can be measured such as audio and vibrations.

One might ask why do we use digital inputs when analog inputs are more general ? Digital inputs are faster, more efficient, and less expensive so they are normally preferred for on/off sensors and actuators. Digital inputs are also less sensitive to noise because there are only two voltages 0V and 5V which are separated well beyond most noise levels.

Note that some of the DIO pins (6 of them) can be used to approximate analog outputs using pulse width modulation (PWM). PWM analog outputs are useful for interfacing with some types of motors such as DC brushed motors in flying drone thrusters. This will be the subject of an Arduino lab for some other course.

The Arduino library uses C++ objects known as “Servo” in order to command the servo motors. If you have not yet formally covered objects in class don’t worry. Programming objects can be challenging, but using them is not so hard. You have probably already used cout and fout which are also objects. In order to use Servo objects you must include <Servo.h> at the top of the program. An object is basically like a new type of variable that has some extra features compared to standard variable types such as int and double. Each servo motor connected to the Arduino requires its own Servo object / variable. For example,

```
#include <Servo.h> // include file for servo motors
```

```
Servo servo1; // declare global Servo object / variable for servo motor #1  
Servo servo2; // declare global Servo object / variable for servo motor #2
```

```
void setup() {  
    servo1.attach(7); // connect pin 7 to servo motor #1  
    servo2.attach(8); // connect pin 8 to servo motor #2  
}
```

```
void loop() {  
    servo1.write(7); // command servo motor #1 to go to 7 deg  
    servo2.write(27); // command servo motor #2 to go to 27 deg  
}
```

This example tells the Arduino that servo1 and servo2 control the servo motors connected to pins 7 and 8, respectively. It then commands servo motor #1 to go to 7 deg and servo motor #2 to go to 27 deg.

Note the expectation for this lab is that you can use Servo objects after studying the examples. You are not expected at this stage to know objects or how to make them. Just use the examples like building blocks / Legos for now.

Programming the Arduino

At this point you can test your setup and run the programming examples. Connect the USB cable to the Arduino and PC.

Lab Tip: If you smell anything like plastic burning then quickly remove the USB connection from the PC. Ask the lab instructor to check your connections in that case.

Expand "arduino_lab_3.rar" given below into a separate folder. Open the "arduino_lab_3.ino" file in the Arduino IDE. After you compile and upload this program the servo motor should go backward and forward and the light sensor output should print out to the screen. The light output should change between 0V and 5V depending on its proximity to light. At this point you can watch the videos and perform the rest of the lab.

This video covers the main programming part of the lab
http://users.encs.concordia.ca/~bwgordon/arduino_lab_3.mp4

This video provides some additional example / exercise problems
http://users.encs.concordia.ca/~bwgordon/arduino_lab_3_examples.mp4

A rar file that contains the source code files for this lesson is provided here
http://users.encs.concordia.ca/~bwgordon/arduino_lab_3.rar

Notes:

The videos may refer to lab #2 since this lab was originally called that. However, the video is for the new version of lab #3.

The lab videos use an Arduino Nano (from a previous lab kit) which is basically the same as the Arduino Uno except smaller. Therefore, the video is equally applicable to the Arduino Uno used in the current lab kit.

The lab videos also use a different version of the servo and light sensor, but the results with the current lab kit should be the same.

Lab assignment

1) The noise level of a sensor can be obtained by repeatedly measuring a sensor for a constant input (e.g. fixed level of light) and calculating statistics for a large number of measurements (e.g. $N = 1000$). The average value is the best estimate of the sensor measurement (assuming the input is not changing). The range delta is given by the difference between the maximum and minimum sensor values measured (i.e. $\text{delta} = \text{max} - \text{min}$). An estimate of the noise level is given by $\text{delta} / 2$. It should be noted that all sensors have some noise and this noise causes uncertainty in measurements. Averaging can be used to reduce the effect of noise if the sensor has a constant or slowly changing input. It can be shown that the noise level of an average measurement is approximated by $\text{delta_ave} = 0.5 * \text{delta} / \sqrt{N}$. Therefore, averaging 25 measurements reduces the effective noise level by a factor of 5.

Determine the average value of the light sensor (for a constant light input) and estimate the noise level of the sensor ($N = 1000$) for measurements made at approximately 1ms intervals. Repeat the test for different levels and types of light (LED, phosphorescent, old fashion light bulbs, etc.). Note any differences in the noise levels and try to explain the variations.

2) Place a small piece of tape on the servo disc so you can clearly see its motion. Then write a program that moves the servo (servo #1 = pin 7) as follows for 60s

$$\text{th1_d} = A * (1.0 + \sin(w * t + \text{phi1})) ;$$

where $A = 45 \text{ deg}$, $w = 1 \text{ rad/s}$, $\text{phi1} = 0$, and t is the time in seconds read from the Arduino clock. The servo motor should move with a sinusoidal motion of 45 degrees in amplitude.

3) Attach the light sensor to the servo disk (using tape, weak glue, etc.). Write a program that maximizes the light to the sensor by adjusting the angle of the servo. Test the program by placing a stationary light (LED, cell phone, etc.) near the sensor. Plot light level vs time in Excel (or another spreadsheet program such as LibreOffice).

Note there are two basic approaches you can use to maximize the light. The first approach slowly moves the servo over its complete range and measures the light for each angle. It then moves to the angle with the highest light measurement. The second approach measures the light for three consecutive angles in a small range ($\text{theta} - d$, theta , and $\text{theta} + d$) a small distance apart (e.g. $d = 3 \text{ deg}$), where theta is the current angle of the servo. The servo then moves to the angle with the highest light level. This process is then continually repeated in order to slowly maximize the light level. Different values of d will result in different convergence times. You can use either approach (or try both) to solve this question. Make sure to give the servo motor enough time to reach its destination. This reaching time will be directly proportional to the distance that must be traveled since the last command. However, any sufficiently large value such as 100 ms will work.